mb-free___transport-eager-8192

An AgentMPI run analysis

Abstract

This run did not complete, and it is the most informative cell in a ten-cell transport sweep for that reason. Eight agent-free ranks were asked to form a ring and send one 8,192-word payload each, with the mode pinned to `Mode.EAGER`; `results/microbench/free-transport.json` records `completed: false`, `n_failed: 8` and `ERR_CONTEXT_OVERFLOW`. The trace has 18 events, not the 34 of every cell that worked, and the sixteen missing events are exactly the eight `msg.send` and eight `msg.recv` events: not one message was ever created. The mechanism is a precondition, not a deadlock. `Communicator._await_eager_credit` compares the payload's 10,779 estimated tokens against the destination's 4,096-token unexpected-message budget and raises before the fabric is touched, so the `messages` and `seqs` tables are empty and the ring never formed a single edge. The detail that makes this the sweep's key cell is that 10,779 tokens would have fitted in the receiver's 32,768-token context budget with room to spare. What refused the message was not the context window but the flow-control credit that protects it, and rendezvous at the same size — same payload, same digest — completed in 34 events with zero context consumed.

1 What this run is

property	value
run	`mb-free__transport-eager-8192`
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	`none` × 8
events	18
wall time	0.04 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.00×	total busy time over wall time
parallel efficiency	0.0%	achieved parallelism per rank
idle fraction	100.0%	rank-seconds paid for and unused
peak ranks busy	0	of 8 in the world
serial fraction of active time	0.0%	time with exactly one rank busy
load imbalance	0.00×	slowest rank's busy time over the mean
coordination share	0.0%	rank-seconds blocked in collectives, of those available
coordination in flight	0.0%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	0	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	0	0 eager, 0 rendezvous
tokens sent	0	0 deferred under rendezvous
tokens in / out	0 / 0	prompt and completion
cost	$0.0000	0.0000 $/kTok out

3 What the analysis notices

Nothing anomalous was detected mechanically.

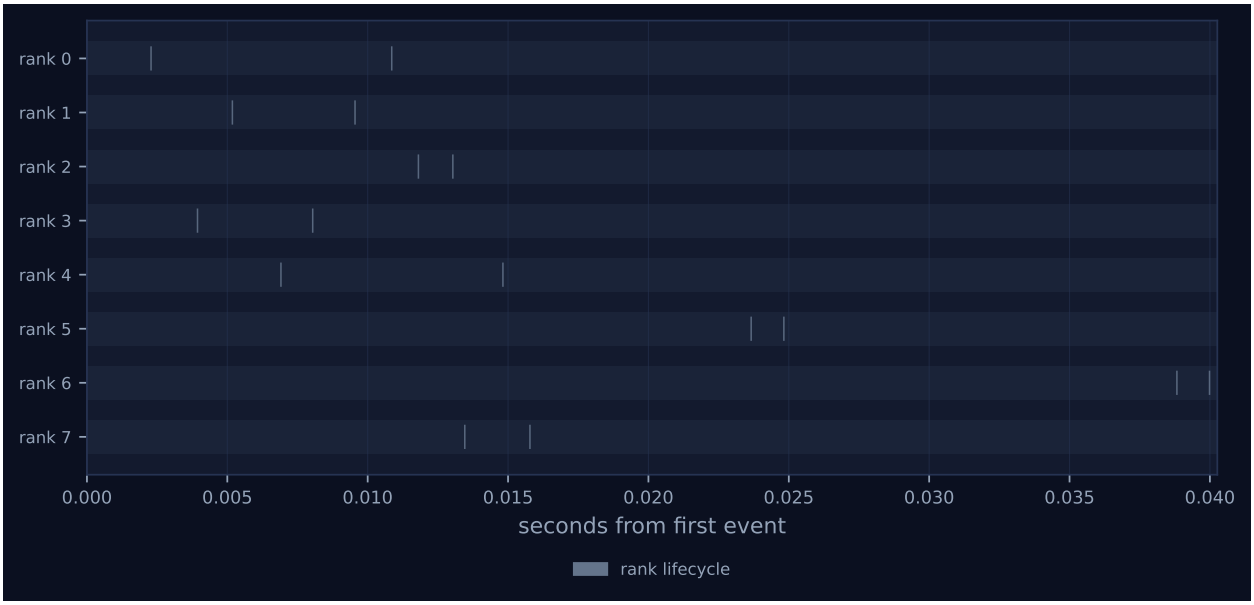

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	0.0	0	0	0	0	0	0.0	finalized
1	0.00	0.0	0	0	0	0	0	0.0	finalized
2	0.00	0.0	0	0	0	0	0	0.0	finalized
3	0.00	0.0	0	0	0	0	0	0.0	finalized
4	0.00	0.0	0	0	0	0	0	0.0	finalized
5	0.00	0.0	0	0	0	0	0	0.0	finalized
6	0.00	0.0	0	0	0	0	0	0.0	finalized
7	0.00	0.0	0	0	0	0	0	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

This run invoked no collectives.

6 Reading the timeline

Figure 1 is the picture of a run that did nothing. Eight lanes, two grey lifecycle diamonds each, and no green anywhere: the trace viewer's legend for this run lists only "rank lifecycle 16" and omits the messages swatch altogether, because there is no message event to colour. The 18 events are `job.create`, eight `rank.init`, eight `rank.finalize` and `job.finish`, and the gap between each rank's two diamonds is 1–6 ms. Every rank came up, attempted its send, and shut down. Compare the 128-, 512- and 2,048-word eager cells, where each lane carries two additional green ticks and `analyze_run.py` draws a communication matrix; here there is no matrix, because there is no traffic to plot, and the figure simply does not exist.

Two things in the dashboard are worth dwelling on, because they are how this failure would be missed. First, the stats row reads `FAILURES 0`, `MESSAGES 0`, and the rank health table gives all eight ranks state `finalized` with 0% occupancy and no suspicion flag. The log agrees: `job.finish` carries `ok: true` and `failed_ranks: []`, and `metrics.json` reports `degraded: false` with zero findings. A run in which all eight ranks raised `AmpiContextOverflow` is recorded, everywhere the trace can be consulted, as a clean job. Second, the span is 0.04 s — *longer* than the 0.035 s of the 2,048-word cell that actually delivered eight messages. That is not noise with a cause worth naming: `Communicator.send` calls `self.fabric.blobs.put(payload)` before it consults the mode or the credit, so the 40 KiB payload was serialised, hashed and written to the content store by every rank, and the blob `3fbdeea17da4...` is on disk in `runs/mb-free/transport-eager-8192/blobs/` at its full 40,960 bytes. The artefact reached the fabric. Only the envelope was refused.

7 Interpretation

The failure path can be identified exactly, and it matters which one it was, because `_await_eager_credit` has two. The reported error class is `ERR_CONTEXT_OVERFLOW`, which is `AmpiContextOverflow`; a receive-side context-budget rejection would have surfaced as `ERR_TRUNCATE` from `ContextAccount.admit`, and there are no agent calls here, so the only remaining source is the send-side credit check. That check begins with a single-payload precondition — if `n_tokens` exceeds the destination's `unexpected_limit` it raises immediately — and continues, if the payload itself is admissible, into a polling loop that waits for outstanding queued tokens to drain and emits `transport.credit_stall` the first time it has to wait. The timing settles which fired. The benchmark passes `timeout=cfg.stall_timeout` = 12 s; the loop would have blocked for that long before raising, and the per-rank wall time recorded in the results file is 0.0 s while the whole job spans 0.04 s. There is no `transport.credit_stall` event in the log. So this was the precondition: 10,779 tokens against a 4,096-token limit, refused in microseconds, 2.63× over. The empty `messages` and `seqs` tables in `fabric.sqlite` confirm that the raise preceded even the sequence-number increment.

The substantive result is which budget did the refusing. The receiver's context budget in this sweep is 32,768 tokens and the payload is 10,779, so the message this cell could not send is one the receiver could comfortably have held — it would have occupied 33% of the window. What blocked it is `unexpected_limit = 4096`, the per-rank budget for messages that have arrived but not yet been received, and that is a flow-control resource rather than a capacity one. AgentMPI is doing here what `comm.py`'s docstring says it intends: MPI implementations either buffer without limit and eventually die of it, or block; this one publishes a bound, refuses a payload that cannot fit under it, and names the remedy in the exception message ("use mode=rendezvous"). The refusal is correct behaviour and it is early, local and attributable. Note also what did *not* happen: the ring is MPI's canonical unsafe program, every rank sending before it receives, and it did not deadlock. It could not have. With $p = 8$ arranged as a ring each destination has exactly one sender, so the aggregate the credit mechanism guards never accumulates, and this sweep exercises the per-message precondition only.

Against its siblings, this is the cell where the eager column stops. `mb-free__transport-eager-2048`, one size down, completed with 2,695 tokens admitted per rank; this one, at 4× the payload, sent nothing. The ceiling therefore lies between 2,048 and 8,192 words, and since the estimator prices `"word "` at a flat 5/3.8 = 1.3158 tokens with no intercept, the crossing is at 3,113 words. Its rendezvous twin, `mb-free__transport-rendezvous-8192`, moves the same 10,779-token payload — the identical content-addressed blob, digest `3fbdeea17da4` — through 34 events in 0.056 s with zero admissions and zero context high water on every rank. The comparison is unusually clean because the two runs differ in nothing except what the receiver is told: same bytes on disk, same eight envelopes, same ring. Eager put the payload in the envelope and was refused; rendezvous put a handle in it and was not. That is the sweep's central claim reduced to a single pair of runs, and this is the half of the pair that fails.

I record one defect, and it is in AgentMPI rather than in the harness. The single-payload rejection path in `_await_eager_credit` emits no event, while the path immediately below it — the one that waits for credit — emits both `transport.credit_stall` and `transport.credit_granted`. The consequence is visible above: a total protocol failure across all eight ranks leaves a trace that is indistinguishable from a job that simply never sent anything, and every derived artefact inherits that. `job.finish` says `ok: true`, `metrics.json` says `degraded: false`, the viewer says `FAILURES 0`, and the only record that this run failed at all lives outside the trace, in the results file. Part of

this is the benchmark's doing — its `rank_main` catches `AmpiError` and returns a dictionary, so the runtime sees a normal return — but the harness could not have logged what the library declined to emit, and the asymmetry between the two rejection paths is the library's. An event on the refusal, carrying `tokens`, `limit` and `dest`, would cost one `fabric.emit` call and would make this run self-describing.

8 Threats to this reading

The strongest threat is that the central fact of this run — that it failed — is not in the trace. `completed: false`, `n_failed: 8` and the error class all come from `results/microbench/free-transport.json` and the trace corroborates them only by absence: 18 events rather than 34, no messages, empty fabric tables. The inference that the precondition branch fired rather than the stalling branch rests on the 0.04 s span against a 12 s stall timeout and on the absence of `transport.credit_stall`, which is strong but is still an inference from what is missing. Second, the location of the ceiling depends on an estimator: provenance records `tokenizer_exact: false`, and 10,779 is $\lceil 40960/3.8 \rceil$ from a character heuristic applied to one repeated word. A real BPE tokeniser would price this payload nearer 8,192 tokens — still 2.0× over the 4,096-token limit, so this cell would still fail, but the crossing point would move from 3,113 words to roughly 4,096. The ceiling's existence is robust; its position in words is not. Third, both budgets in this run are benchmark parameters, not defaults: `unexpected_limit` is 4,096 here against a shipped default of 8× the eager limit, and `context_budget` is 32,768 against a default of 128,000. A production harness with the shipped defaults would place the cliff somewhere else entirely, and this run says where AgentMPI puts a cliff when asked to, not where the cliff is. And because the executor is `none` with zero agent calls, nothing here bears on agent behaviour: the run is one sample of a protocol decision taken in microseconds, and its wall time is SQLite writes.